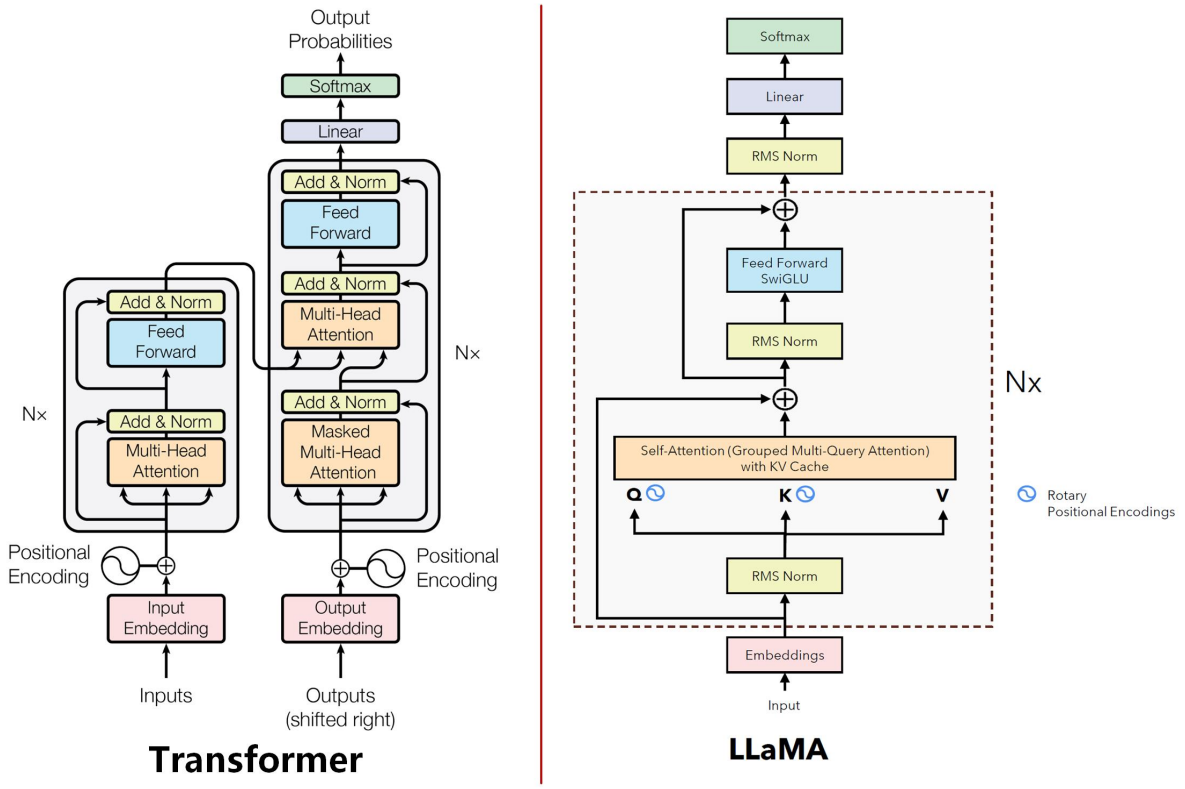
Llama 3模型架构



Llama 3模型架构

Llama 3模型基于标准的Transformer架构进行了多项改进，包括更高的效率和更好的性能。

接下来，我们详细探讨Llama 3架构的主要特点。

模型概述

- **模型类型：** 基于解码器的Transformer
- **参数量：** 8B和70B
- **上下文长度：** 8192（LLaMA-1和LLaMA-2的上下文长度分别为2048, 4096）
- **注意力机制：** 分组查询注意力（Grouped Query Attention, GQA）

结构组成

Llama 3的模型架构主要包括以下组件：

分层堆叠

- **嵌入层：** 将输入的token转换为固定维度的嵌入表示。
- **自注意力层：** 包含多头自注意力机制和归一化。
- **前馈网络（FFN）：** 包含激活函数和两层全连接网络。
- **位置编码：** 采用RoPE（Rotary Position Embedding）位置编码。

Llama 3与传统Transformer架构的比较

相同之处

- **基本结构：** 两者都基于标准的Transformer解码器架构。
- **多头自注意力：** 都使用多头自注意力机制。
- **前馈网络（FFN）：** 包含两层全连接网络。

不同之处

1. **注意力机制：**

- **传统Transformer：** 使用标准多头自注意力机制。
  - **Llama 3：** 引入了分组查询注意力（GQA）以提高效率。

2. **激活函数：**
   - **传统Transformer：** 使用ReLU或GELU激活函数。
   - **Llama 3：** 使用更高效的SwiGLU激活函数。
3. **位置编码：**
   - **传统Transformer：** 使用正弦-余弦位置编码或Learned Positional Embedding。
   - **Llama 3：** 使用RoPE（Rotary Position Embedding）位置编码。
4. **上下文长度：**
   - **传统Transformer：** 通常为512或1024的上下文长度。
   - **Llama 3：** 上下文长度增加到8192。

LLama3模型组件详解

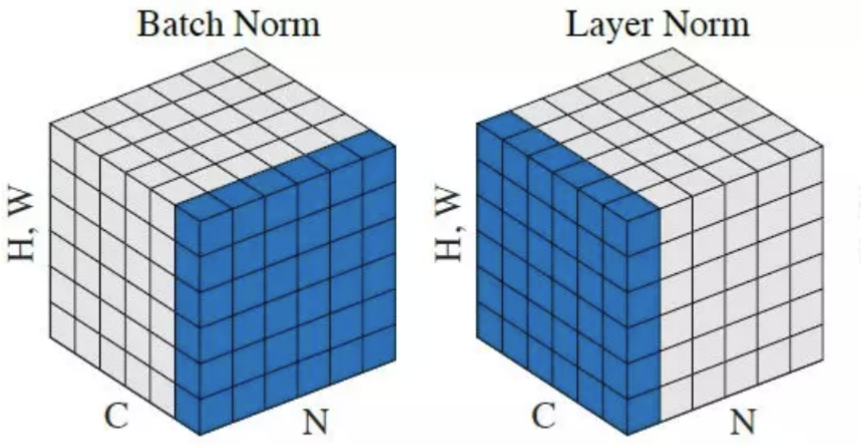
RMSNorm

归一化技术是一种在神经网络中用于归一化激活值的技术。

它可以提高训练的稳定性，加快收敛速度并改善模型的泛化性能。

下面是Transformer架构中常见的两种归一化技术：

层归一化（Layer Normalization）



层归一化是Transformer架构中最早引入的归一化技术之一。

它将一个输入序列的所有特征归一化，使得每个样本的激活值在特征维度上具有均值为0、方差为1的分布。其公式如下：

$$\hat{x}_i = \frac{x_i - \mu}{\sigma + \epsilon}$$

其中，μ 是均值，σ 是标准差，ϵ 是一个很小的数以避免除以零。

RMSNorm

RMSNorm是一种简化版的层归一化，它通过均方根（Root Mean Square）计算得到归一化尺度，不需要计算均值和标准差。

其公式如下：

$$\hat{x}_i = \frac{x_i}{\sqrt{\frac{1}{n}\sum_{j=1}^{n} x_j^2 + \epsilon}}$$

与层归一化相比，RMSNorm减少了计算复杂度，同时保持了良好的归一化效果。

RMSNorm在LLaMA中的应用

在LLaMA模型中，RMSNorm主要用于对每个时间步的隐藏状态向量进行归一化。

每个时间步的输入向量被独立归一化，适用于处理变长序列和保持时间步间的独立性。

具体来说，RMSNorm在Transformer架构中的应用通常在以下几个位置：

1. **多头自注意力机制的输出：**
   - 对于每个时间步的隐藏状态，在经过多头自注意力机制后的输出进行归一化。
2. **前馈神经网络的输出：**
   - 对于每个时间步的隐藏状态，在经过前馈神经网络（Feed-Forward Neural Network, FFN）后的输出进行归一化。

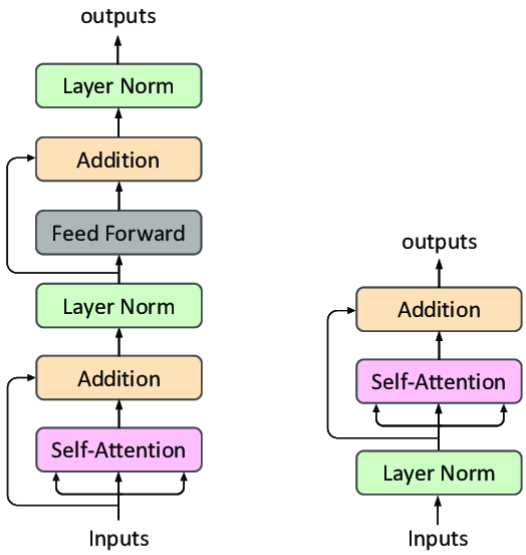
为什么选择RMSNorm?

在LLaMA中，选择RMSNorm作为主要的归一化策略，原因如下：

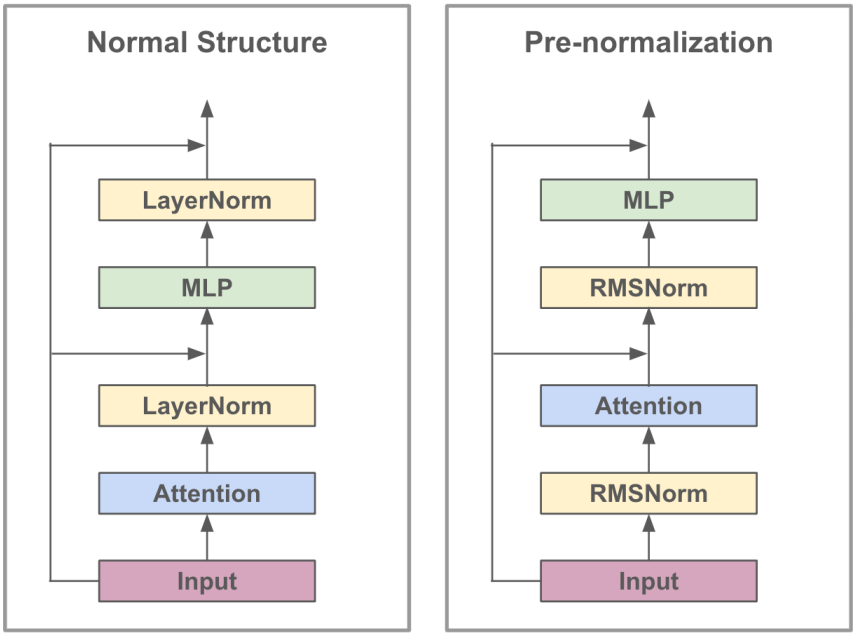
1. **提高训练效率**：RMSNorm在计算上比层归一化更高效。在大型语言模型中，效率的提升尤其显著。
2. **提高训练稳定性和泛化性能**：研究发现，RMSNorm可以提供更稳定的训练过程，减少梯度爆炸或消失的风险，同时保持较高的泛化能力。
3. **预归一化的优势**：LLaMA采用了RMSNorm的预归一化变体，将归一化操作放在Transformer模块的主要层之前，而不是之后。

   这一策略进一步提升了模型的训练稳定性。

51CTO学堂



(a) Original transformer　　(b) Pre-layernorm transformer



RMSNorm vs LayerNorm的对比

LLaMA中的实验结果显示，RMSNorm在效率和性能方面都有明显的提升：

- **效率提升**：与层归一化相比，RMSNorm在效率上提升了10-50%。
- **性能对比**：在实际应用中，RMSNorm在稳定性和泛化性能上与层归一化不相上下，有时甚至更优。

SwiGLU激活函数

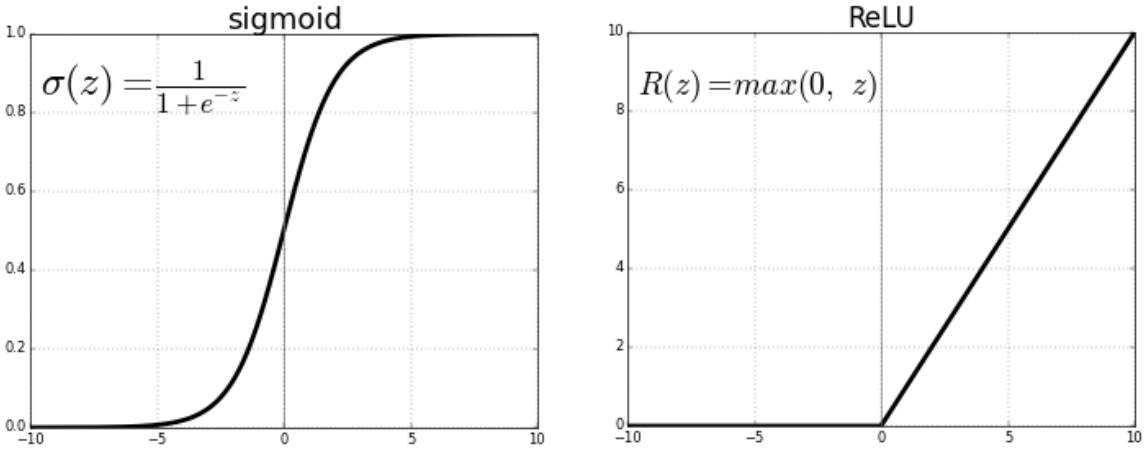
激活函数在大型语言模型（LLM）的发展过程中扮演着关键角色，它影响着模型性能的稳定性与泛化能力。

LLaMA3在其前馈层中采用了一种特殊的激活函数——SwiGLU。

激活函数是深度神经网络中的关键组件，负责引入非线性，使模型能够学习复杂的数据模式。

常见的激活函数包括：

ReLU（Rectified Linear Unit）



ReLU是目前使用最广泛的激活函数之一。其计算公式如下：

$$\mathrm{ReLU}(x) = \max(0, x)$$

特点：

- 计算简单，梯度为0或1。
- 可能导致"神经元死亡"问题。

GELU（Gaussian Error Linear Unit）

GELU是一种基于高斯分布的激活函数，在GPT-3等模型中使用。

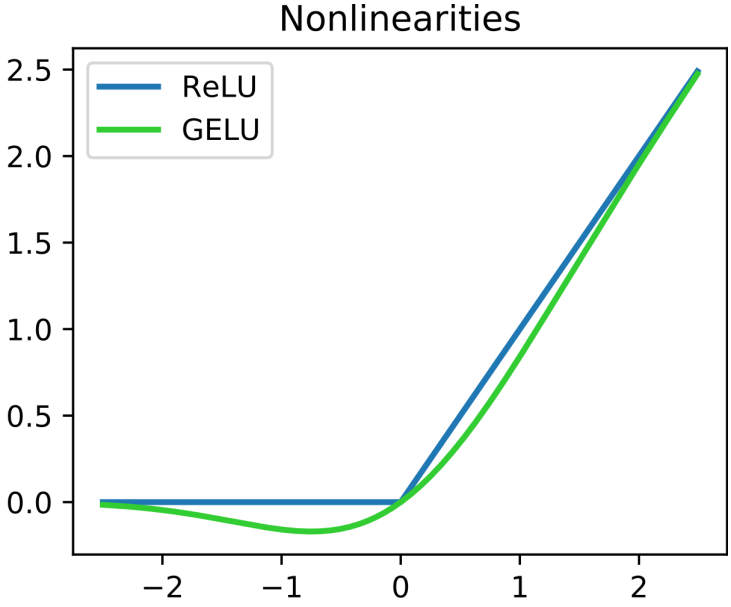
其公式如下：

$$\mathrm{GELU}(x) = x \cdot 0.5 \cdot (1 + \mathrm{erf}(\frac{x}{\sqrt{2}}))$$

其中，erf为误差函数。

特点：

- 与ReLU相比，GELU具有平滑的激活曲线，能提供更高的性能。
-



Swish

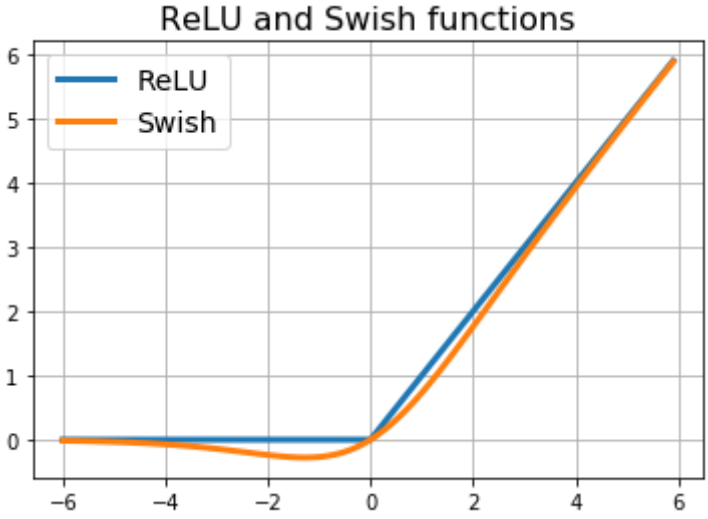
Swish是一种平滑且连续的激活函数，在Transformer等模型中应用广泛。其公式如下：

$$\text{Swish}(x) = x \cdot \sigma(x)$$

其中，$\sigma(x) = \frac{1}{1+e^{-x}}$ 为标准Sigmoid函数。

特点：

- 平滑的曲线有助于稳定梯度流。
- 通常比ReLU表现更好。



GLU（Gated Linear Unit）

GLU（Gated Linear Unit）是一种用于神经网络中的激活函数，最初由 Yann Dauphin 等人在论文《Language Modeling with Gated Convolutional Networks》里提出。

GLU 的设计灵感来自门控机制，它通过引入门控操作来控制信息的流动。

该操作可以看作是引入了一种动态的选择机制，以在模型中选择性地传递信息。

GLU 的计算公式如下：

$$\text{GLU}(X) = (X \cdot W_1 + b_1) \odot \sigma(X \cdot W_2 + b_2)$$

其中：

- X 是输入特征。
- W_1, W_2 是两个不同的权重矩阵。
- b_1, b_2 是偏置。
- σ 是 Sigmoid 函数，作为门控信号的激活函数。
- $\odot$ 表示元素级别的乘法（Hadamard 积）。

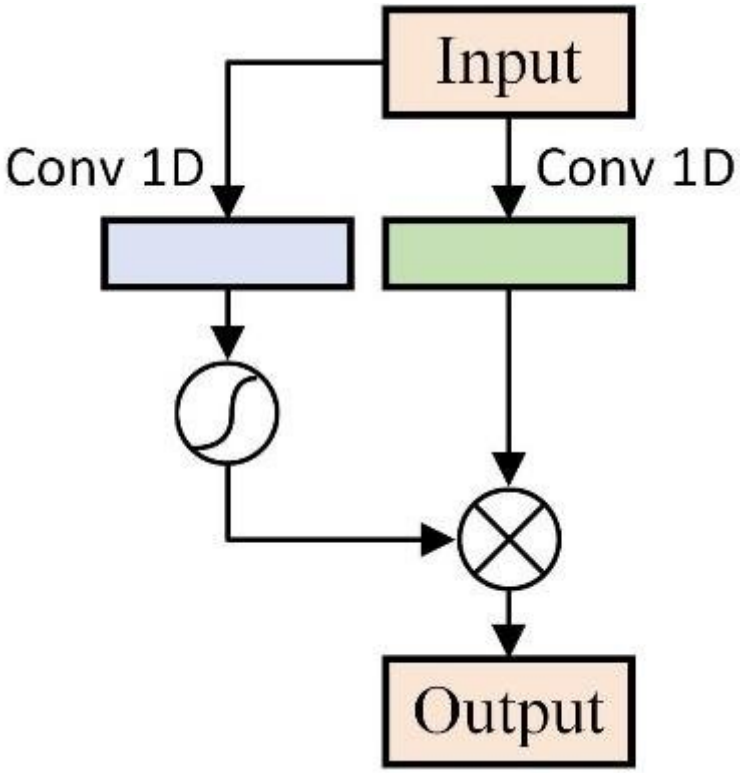
在 GLU 中，输入特征 X 被分成两个部分：

1. **线性部分：** $X \cdot W_1 + b_1$
2. **门控部分：** $\sigma(X \cdot W_2 + b_2)$

门控部分使用 Sigmoid 函数输出一个介于 0 和 1 之间的权重矩阵，从而选择性地让部分信息通过。

因此，GLU 实际上是在学习一种信息过滤机制，以根据输入数据的特征动态调整信息的流动。

51CTO学堂



优势

- **动态信息选择：** 门控机制允许模型根据输入特征选择性地传递信息，提高模型的灵活性。
- **性能提升：** GLU 在自然语言处理任务中证明可以比传统激活函数（如 ReLU）获得更好的性能。

SwiGLU激活函数

SwiGLU（Swish-Gated Linear Unit）是一种结合Swish激活函数与GLU（Gated Linear Unit）机制的激活函数。其公式如下：

$$\text{SwiGLU}(x) = (\text{Swish}(W_1 x)) \cdot (W_2 x)$$

其中：

- W_1 和 W_2 分别是输入 x 的两个线性变换矩阵。
- $\text{Swish}(z) = z \cdot \sigma(z)$，其中 $\sigma(z)$ 是标准的Sigmoid函数。

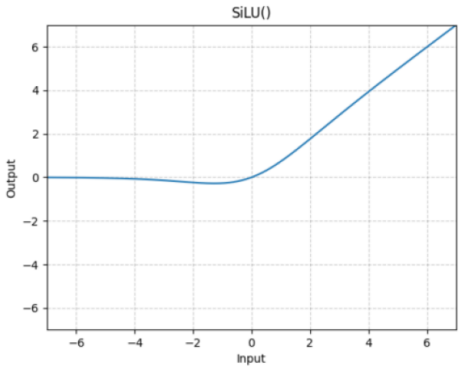
该公式可以理解为：

1. 对输入 x 进行两次线性变换。
2. 将其中一个结果通过Swish激活函数。
3. 将两个结果逐元素相乘，形成最终输出。

51CTO学堂

We use the swish function with $\beta = 1$. In this case it's called the **Sigmoid Linear Unit** (**SiLU**) function.

$$\text{swish}(x) = x\,\text{sigmoid}(\beta x) = \frac{x}{1 + e^{-\beta x}}$$



SwiGLU的计算代价与性能优势

```
class SwiGLU(nn.Module):

    def __init__(self, w1, w2, w3) -> None:
        super().__init__()
        self.w1 = w1
        self.w2 = w2
        self.w3 = w3

    def forward(self, x):
        x1 = F.linear(x, self.w1.weight)
        x2 = F.linear(x, self.w2.weight)
        hidden = F.silu(x1) * x2
        return F.linear(hidden, self.w3.weight)
```

SwiGLU激活函数需要进行三次矩阵乘法运算，相比于ReLU等传统激活函数计算复杂度更高。

然而，研究发现，尽管计算量增加，SwiGLU带来的性能提升显著：

- **更好的泛化性能**：SwiGLU在处理复杂的文本数据时表现出更好的泛化能力。
- **稳定的梯度流**：Swish激活函数平滑的曲线特性有助于稳定梯度，减少梯度消失和爆炸的可能性。

RoPE旋转位置编码

位置编码是Transformer中确保模型能够理解序列顺序信息的重要部分。

传统的绝对和相对位置编码方案各有优缺点，然而RoPE（Rotary Position Embedding）作为一种新型的位置编码方法，平衡了绝对和相对位置编码的优点。

位置编码的背景

位置编码在Transformer中至关重要，因为自注意力机制本质上是无序的。

常见的两种位置编码方法是：

- **绝对位置编码（Absolute Positional Encoding）**：为序列中的每个位置提供一个固定的嵌入。
- **相对位置编码（Relative Positional Encoding）**：表示序列中每两个token之间的相对位置信息。

51CTO学堂

绝对位置编码的特性

绝对位置编码是指在序列中的每个位置直接关联一个固定的嵌入。

例如，Transformer中常用的正弦-余弦绝对位置编码如下：

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right)$$
$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

其中，`pos`表示序列中的位置，`d`表示嵌入维度。

这种方法通过给定的位置索引为每个位置提供唯一的编码，确保模型能够理解token之间的顺序。

相对位置编码的特性

相对位置编码关注的是序列中两个token之间的相对距离，而非绝对位置。

相对位置编码可以帮助模型在不同长度的输入序列之间共享信息，具体实现方式多种多样。例如，在Transformer-XL模型中：

$$A_{ij} = Q_i \cdot K_j + Q_i \cdot r_{i-j}$$

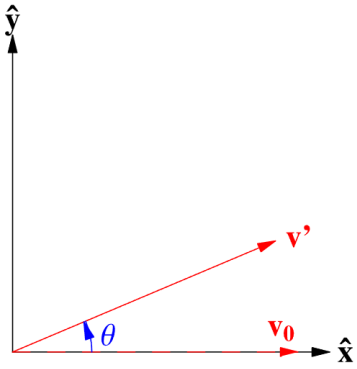
其中，`r_{i-j}`表示相对距离的嵌入，确保模型对不同长度的输入都能够表现良好。

RoPE的原理

RoPE（Rotary Position Embedding）结合了绝对和相对位置编码的优点。

它使用旋转矩阵对每个位置进行编码，并直接将相对位置信息引入自注意力操作中。

旋转矩阵(Rotation Matrix)



In $\mathbb{R}^2$, consider the matrix that rotates a given vector $\mathbf{v}_0$ by a counterclockwise angle θ in a fixed coordinate system. Then

$$\mathsf{R}_\theta = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix},$$

so

$$\mathbf{v}' = \mathsf{R}_\theta \mathbf{v}_0.$$

RoPE的数学公式

假设输入向量为 $x \in \mathbb{R}^d$，其第 i 个位置的编码向量可以表示为：

$$x_i' = \mathrm{RoPE}(x_i, i)$$

其中：

51CTO学堂

$$\text{RoPE}(x_i, i) = R(i)x_i$$

这里的旋转矩阵 $R(i)$ 用二维旋转矩阵的张量乘积进行定义。

将向量 x_i 拆分为一系列长度为2的子向量 $(x_{i,2k}, x_{i,2k+1})$，

其旋转形式为：

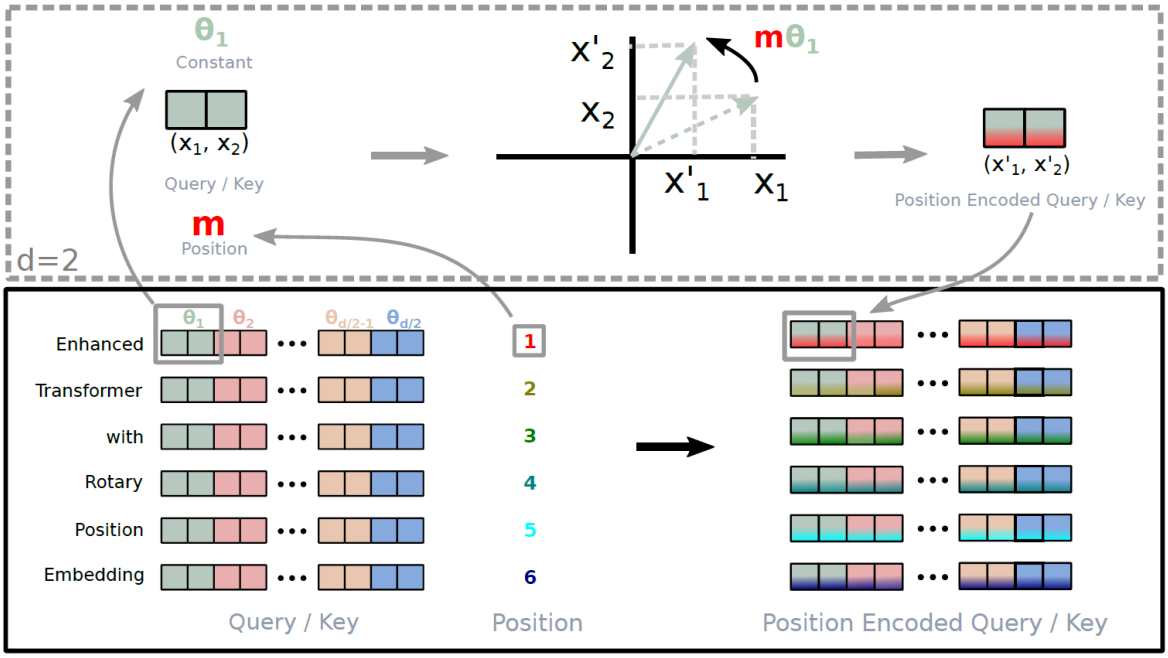
$$R(i)\begin{bmatrix} x_{i,2k} \\ x_{i,2k+1} \end{bmatrix} = \begin{bmatrix} \cos(\theta_k i) & -\sin(\theta_k i) \\ \sin(\theta_k i) & \cos(\theta_k i) \end{bmatrix} \begin{bmatrix} x_{i,2k} \\ x_{i,2k+1} \end{bmatrix}$$

其中：

$$\theta_k = \frac{10000^{-2k/d}}{\pi}$$

经过RoPE编码后的输入向量与旋转矩阵结合，使得位置信息被直接嵌入到输入向量中。

$$\boldsymbol{R}^d_{\Theta,m} = \begin{pmatrix} \cos m\theta_1 & -\sin m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ \sin m\theta_1 & \cos m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ 0 & 0 & \cos m\theta_2 & -\sin m\theta_2 & \cdots & 0 & 0 \\ 0 & 0 & \sin m\theta_2 & \cos m\theta_2 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \cdots & \cos m\theta_{d/2} & -\sin m\theta_{d/2} \\ 0 & 0 & 0 & 0 & \cdots & \sin m\theta_{d/2} & \cos m\theta_{d/2} \end{pmatrix}$$



RoPE的相对位置编码特性

在自注意力计算中，RoPE能够引入相对位置信息。

令 q_i 和 k_j 分别表示位置 i 和 j 处的查询和键向量，经过RoPE编码后，其点积为：

$$\langle q'_i, k'_j \rangle = \langle R(i)q_i, R(j)k_j \rangle = \langle q_i, R(j-i)k_j \rangle$$

这意味着，通过RoPE编码，查询和键向量之间的相对位置信息直接被融入自注意力操作中。

51CTO 学堂

RoPE（Rotary Position Embedding）结合了绝对和相对位置编码的优点，因为它既能够编码绝对信息，又能够在自注意力操作中有效融入相对位置信息。

RoPE如何结合绝对和相对位置编码

RoPE通过旋转矩阵为每个位置编码，实现了绝对位置信息的嵌入，同时在自注意力操作中有效地引入了相对位置信息。

RoPE的编码过程

给定一个嵌入维度为 `d` 的输入向量 $x \in \mathbb{R}^d$，其位置编码可以表示为：

$$x_i' = \text{RoPE}(x_i, i) = R(i) \cdot x_i$$

其中，旋转矩阵 $R(i)$ 用二维旋转矩阵的张量乘积形式定义。

假设 x_i 可以分解为一系列长度为2的子向量 $(x_{i,2k}, x_{i,2k+1})$，其旋转矩阵为：

$$R(i) \begin{bmatrix} x_{i,2k} \\ x_{i,2k+1} \end{bmatrix} = \begin{bmatrix} \cos(\theta_k \cdot i) & -\sin(\theta_k \cdot i) \\ \sin(\theta_k \cdot i) & \cos(\theta_k \cdot i) \end{bmatrix} \begin{bmatrix} x_{i,2k} \\ x_{i,2k+1} \end{bmatrix}$$

其中：

$$\theta_k = \frac{10000^{-2k/d}}{\pi}$$

这个矩阵表示每个位置 i 的绝对编码。

RoPE引入相对位置编码

经过RoPE编码的查询和键向量，在自注意力机制中的点积计算如下：

$$\langle q_i', k_j' \rangle = \langle R(i)q_i, R(j)k_j \rangle = \langle q_i, R(j-i)k_j \rangle$$

其中：

$$R(j-i) = \begin{bmatrix} \cos(\theta_k \cdot (j-i)) & -\sin(\theta_k \cdot (j-i)) \\ \sin(\theta_k \cdot (j-i)) & \cos(\theta_k \cdot (j-i)) \end{bmatrix}$$

这意味着RoPE在计算查询和键之间的点积时，将相对位置信息直接融入了自注意力操作中。

RoPE的优点

1. **引入相对位置信息**：RoPE能够在自注意力操作中直接编码相对位置信息，使模型具有较好的相对位置感知能力。
2. **保持绝对位置信息**：RoPE的旋转矩阵编码每个位置的绝对信息，保持绝对位置感知能力。
3. **高效处理长序列**：在长序列任务中，RoPE相较于其他位置编码方案表现出更高的效率和性能。

RoPE通过旋转矩阵将绝对位置与相对位置信息相结合，既提供了绝对位置信息的精确性，又具备相对位置感知的灵活性，成为LLaMA等大型语言模型中重要的位置编码方法。

RoPE的优化与实现

51CTO 学堂

Since the matrix is **sparse**, it is not convenient to use it to compute the positional embedd

$$f_{\{q,k\}}(\boldsymbol{x}_m, m) = \boldsymbol{R}^d_{\Theta,m}\boldsymbol{W}_{\{q,k\}}\boldsymbol{x}_m \tag{14}$$

where

$$\boldsymbol{R}^d_{\Theta,m} = \begin{pmatrix} \cos m\theta_1 & -\sin m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ \sin m\theta_1 & \cos m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ 0 & 0 & \cos m\theta_2 & -\sin m\theta_2 & \cdots & 0 & 0 \\ 0 & 0 & \sin m\theta_2 & \cos m\theta_2 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \cdots & \cos m\theta_{d/2} & -\sin m\theta_{d/2} \\ 0 & 0 & 0 & 0 & \cdots & \sin m\theta_{d/2} & \cos m\theta_{d/2} \end{pmatrix} \tag{15}$$

is the rotary matrix with pre-defined parameters $\Theta = \{\theta_i = 10000^{-2(i-1)/d}, i \in [1, 2, ..., d/2]\}$. A graphic illustration of RoPE is shown in Figure (1). Applying our RoPE to self-attention in Equation (2), we obtain:

$$\boldsymbol{q}_m^\intercal \boldsymbol{k}_n = (\boldsymbol{R}^d_{\Theta,m}\boldsymbol{W}_q\boldsymbol{x}_m)^\intercal(\boldsymbol{R}^d_{\Theta,n}\boldsymbol{W}_k\boldsymbol{x}_n) = \boldsymbol{x}^\intercal\boldsymbol{W}_q R^d_{\Theta,n-m}\boldsymbol{W}_k\boldsymbol{x}_n \tag{16}$$

where $\boldsymbol{R}^d_{\Theta,n-m} = (\boldsymbol{R}^d_{\Theta,m})^\intercal\boldsymbol{R}^d_{\Theta,n}$. Note that $\boldsymbol{R}^d_\Theta$ is an orthogonal matrix, which ensures stability during the process of encoding position information. In addition, due to the sparsity of R^d_Θ, applying matrix multiplication directly as in Equation (16) is not computationally efficient; we provide another realization in theoretical explanation.

优化策略

1. **分块并行处理**：RoPE可以通过分块并行化矩阵运算来提高效率。
2. **预计算旋转矩阵**：对于固定序列长度的任务，可以预计算并缓存旋转矩阵，减少实时计算量。

- Given a token with embedding vector **x**, and the position **m** of the token inside the sentence, this is how we compute the position embeddings for the token.

3.4.2 Computational efficient realization of rotary matrix multiplication

Taking the advantage of the sparsity of $\boldsymbol{R}^d_{\Theta,m}$ in Equation (15), a more computational efficient realization of a multiplication of R^d_Θ and $\boldsymbol{x} \in \mathbb{R}^d$ is:

$$\boldsymbol{R}^d_{\Theta,m}\boldsymbol{x} = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ \vdots \\ x_{d-1} \\ x_d \end{pmatrix} \otimes \begin{pmatrix} \cos m\theta_1 \\ \cos m\theta_1 \\ \cos m\theta_2 \\ \cos m\theta_2 \\ \vdots \\ \cos m\theta_{d/2} \\ \cos m\theta_{d/2} \end{pmatrix} + \begin{pmatrix} -x_2 \\ x_1 \\ -x_4 \\ x_3 \\ \vdots \\ -x_{d-1} \\ x_d \end{pmatrix} \otimes \begin{pmatrix} \sin m\theta_1 \\ \sin m\theta_1 \\ \sin m\theta_2 \\ \sin m\theta_2 \\ \vdots \\ \sin m\theta_{d/2} \\ \sin m\theta_{d/2} \end{pmatrix} \tag{34}$$

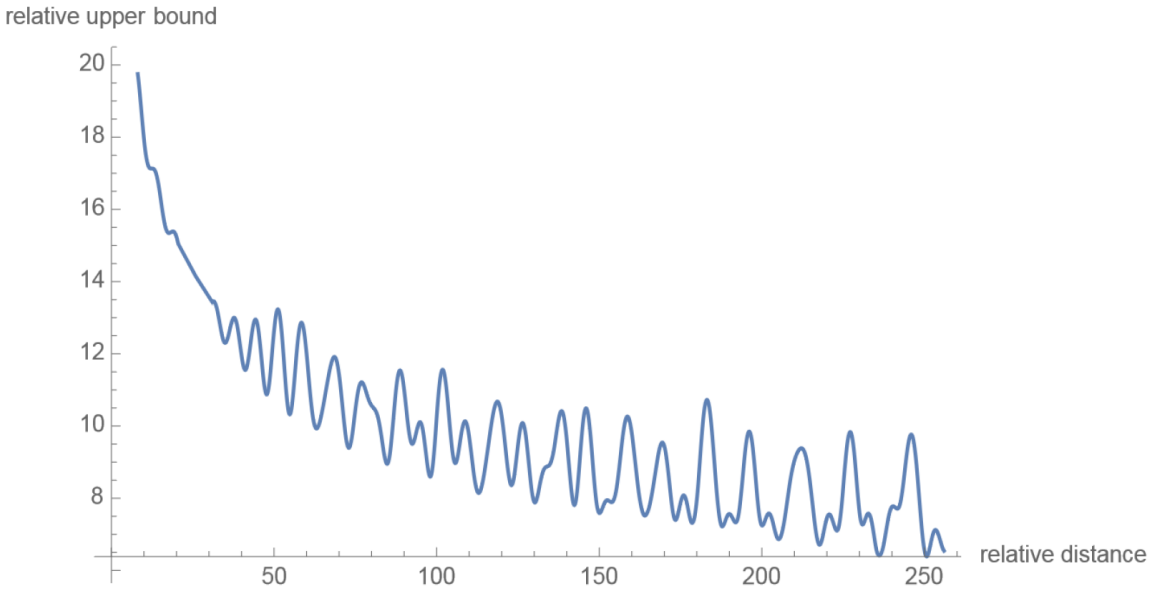


Figure 2: Long-term decay of RoPE.

51CTO学堂

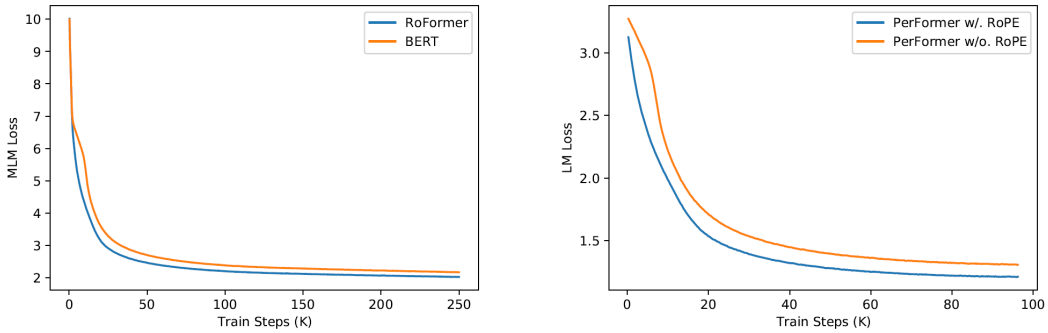


Figure 3: Evaluation of RoPE in language modeling pre-training. **Left**: training loss for BERT and RoFormer. **Right**: training loss for PerFormer with and without RoPE.

分组查询注意力（GQA）

分组查询注意力（Grouped Query Attention，简称GQA）作为一种改进的多头自注意力机制，最近在Llama 3模型中得到了应用。

GQA旨在平衡多头自注意力和多查询注意力的优缺点，以提高推理效率并保持高性能。

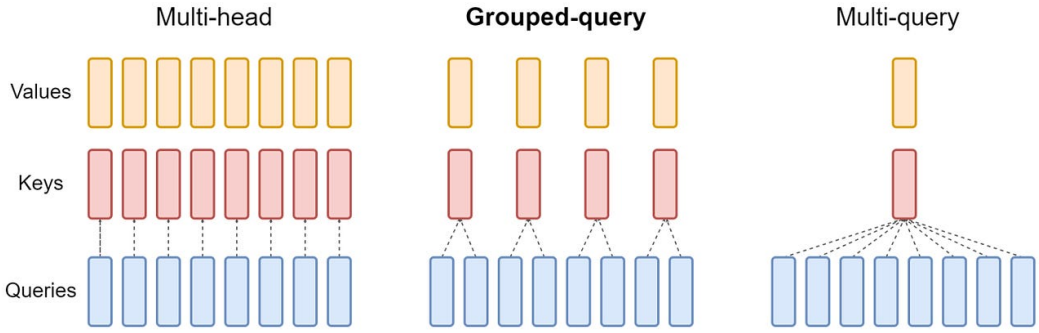


Figure 2: Overview of grouped-query method. Multi-head attention has H query, key, and value heads. Multi-query attention shares single key and value heads across all query heads. Grouped-query attention instead shares single key and value heads for each *group* of query heads, interpolating between multi-head and multi-query attention.

多头自注意力机制（Multi-Head Self-Attention）

多头自注意力机制是Transformer模型的核心组件，旨在通过并行化的方式捕获序列中的复杂关系。其计算公式如下：

1. **查询（Query）、键（Key）和值（Value）向量：**

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

其中，W_Q、W_K 和 W_V 分别是查询、键和值的投影矩阵。

2. **自注意力计算：**

$$A = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

其中，d_k 为查询和键向量的维度。

3. **多头组合：**

$$\text{Multi-Head}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W_O$$

其中，$\text{head}_i = \text{Attention}(QW_{Q_i}, KW_{K_i}, VW_{V_i})$，$W_O$ 是输出矩阵。

多查询注意力机制（Multi-Query Attention）

多查询注意力机制简化了多头自注意力，将所有注意力头共享相同的键和值投影，从而减少计算和内存开销。其结构如下：

1. **查询向量：**

$$Q = XW_Q$$

2. **共享的键和值向量：**

$$K = XW_K, \quad V = XW_V$$

3. **自注意力计算：**

$$A_i = \text{softmax}\left(\frac{QW_{Q_i}K^T}{\sqrt{d_k}}\right)V$$

4. **组合：**

$$\text{Multi-Query}(Q, K, V) = \text{Concat}(A_1, \ldots, A_h)W_O$$

GQA（Grouped Query Attention）的原理

GQA结合了多头自注意力和多查询注意力的优点，通过分组的方式共享键和值投影。

其结构如下：

1. **查询向量：**

$$Q = XW_Q$$

2. **分组的键和值向量：**

将总计 N 个注意力头划分为 G 组，每组共享相同的键和值投影：

$$K_g = XW_{K_g}, \quad V_g = XW_{V_g}, \quad g = 1, \ldots, G$$

3. **组内自注意力计算：**

$$A_{g,i} = \text{softmax}\left(\frac{QW_{Q_{g,i}}K_g^T}{\sqrt{d_k}}\right)V_g$$

4. **组合：**

$$\text{Grouped-Query}(Q, K, V) = \text{Concat}(A_{1,1}, \ldots, A_{G,h/G})W_O$$

其中：

- $W_{Q_{g,i}}$ 为每个查询头的投影矩阵。
- W_{K_g} 和 W_{V_g} 为每组共享的键和值投影矩阵。

GQA的优势

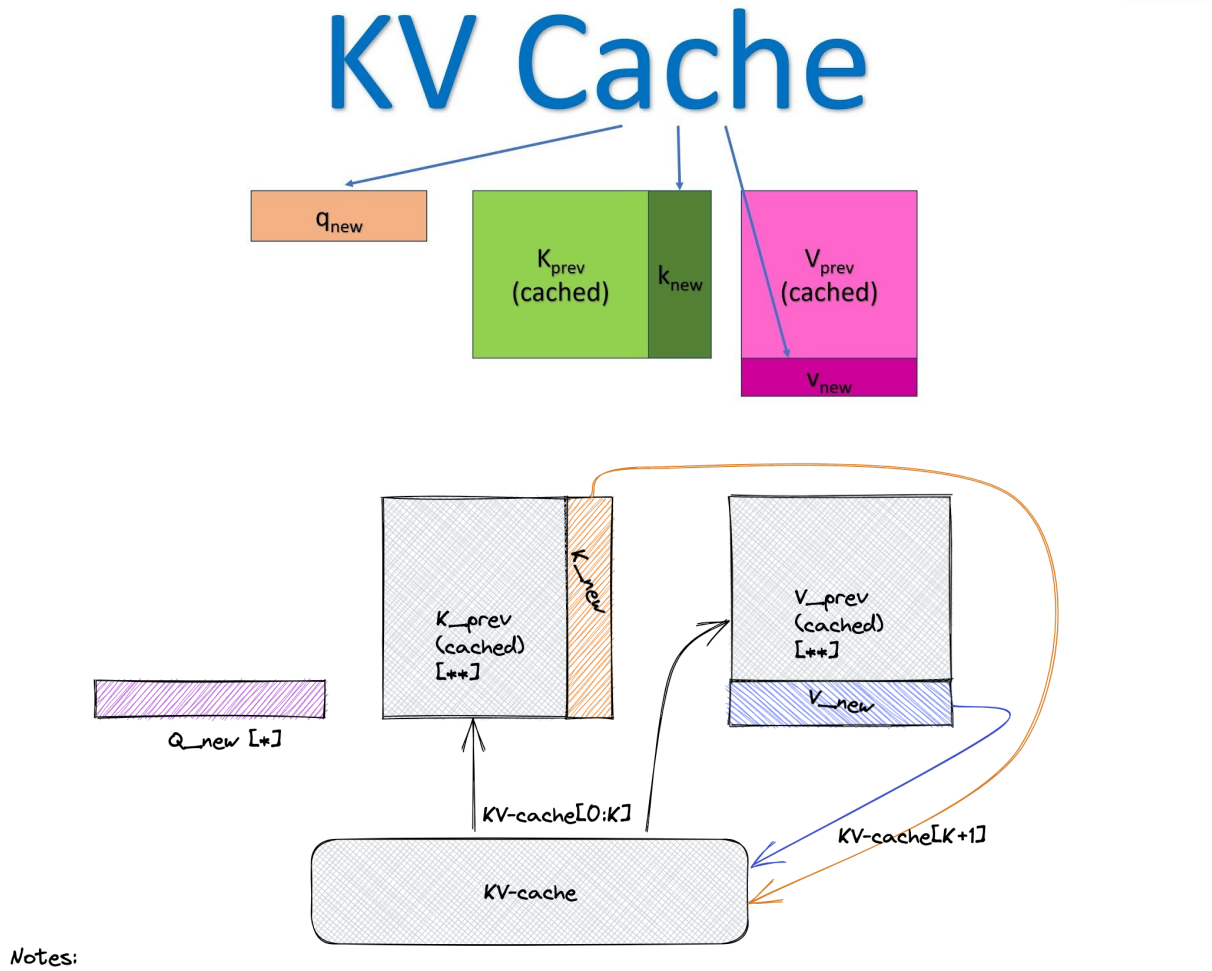
- **计算效率：** 减少了键和值的计算和内存开销。
- **性能保持：** 在性能上与多头自注意力相当，同时在效率上接近多查询注意力。

KVCache

在大型语言模型（LLM）的推理和训练过程中，缓存机制对于提高模型的效率和性能至关重要。

KVCache（Key-Value Cache）是一种用于缓存自注意力机制中键和值的优化策略，可以显著加速推理过程。

51CTO学堂



自注意力机制中的键和值

在Transformer的自注意力机制中，键和值的计算是核心步骤：

1. **输入向量投影：**
   - 查询向量 $Q = XW_Q$
   - 键向量 $K = XW_K$
   - 值向量 $V = XW_V$
2. **自注意力计算：**

$$A = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

其中，d_k 为键向量的维度。

多头自注意力机制

多头自注意力机制将上述计算扩展为并行计算多个注意力头：

$$\text{Multi-Head}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W_O$$

每个注意力头使用不同的投影矩阵，捕获序列中的不同关系。

推理过程中的重复计算

在推理过程中，模型可能会接收一系列的连续输入。

对于每个新的输入，模型会重新计算键和值，导致大量的重复计算。

这种情况在长文本生成或对话任务中尤其明显。

KVCache的原理

KVCache旨在减少自注意力机制中的重复计算。

它通过缓存先前计算的键和值，在推理过程中复用这些信息，从而显著提高推理效率。

KVCache的工作流程

1. **初始化缓存：** 初始化一个空的键和值缓存。
2. **计算当前输入的键和值：**

$$K_t = X_t W_K, \quad V_t = X_t W_V$$

3. **更新缓存：** 将当前输入的键和值追加到缓存中。
4. **自注意力计算：** 在新的键和值上进行自注意力计算：

$$A = \text{softmax}\left(\frac{Q K_{\text{cache}}^T}{\sqrt{d_k}}\right) V_{\text{cache}}$$

5. **清理缓存：**

在某些情况下（如达到上下文长度上限），需要清理缓存或进行部分替换。

KVCache在Llama 3中的应用

Llama 3作为Meta发布的最新大型语言模型，采用了KVCache机制以提高推理效率。

Llama 3通过引入KVCache机制，将推理过程中的键和值缓存下来，在后续的注意力计算中复用这些缓存，以减少重复计算。

- **键和值缓存：** 在每一层的自注意力机制中，键和值的计算结果都会被缓存。
- **缓存更新策略：** 每次新的输入到来时，缓存会根据新的键和值进行更新，确保自注意力计算的准确性。
- **缓存清理：** 当达到上下文长度上限时，缓存会被部分或全部清理。

KVCache作为一种优化策略，通过缓存自注意力机制中的键和值，有效减少了推理过程中的重复计算。

LLM文本生成的两个阶段：预填充（prefill）和解码（decode）

在大型语言模型（LLM）生成文本的过程中，通常会涉及两个阶段：预填充（prefill）和解码（decode）。

预填充（Prefill）阶段

预填充阶段主要用于准备初始上下文，为模型生成后续文本提供基础。

在这个阶段，模型会处理输入的初始文本（通常是用户提供的提示或上下文），并生成相应的内部状态（如KV缓存）。

具体过程

1. **初始上下文输入：**
    - 用户提供一个初始文本作为提示，例如问题、句子或段落。

- 该文本通常被编码为一系列token（tokens）。

2. **初始上下文的注意力计算：**
   - 模型对输入的所有token进行处理，生成键和值，并存储在KV缓存中。
3. **初始概率分布：**
   - 计算最后一个token的概率分布，作为生成下一个token的起始点。

解码（Decode）阶段

解码阶段是模型根据预填充阶段准备好的上下文，生成后续文本的过程。

在这个阶段，模型会逐步生成每个新token，并在每次生成后更新KV缓存。

具体过程

1. **token生成：**
   - 从预填充阶段的初始概率分布中采样一个新token，作为下一个输入。
   - 根据用户选择的策略（如贪心、采样、温度）进行采样。
2. **KV缓存更新：**
   - 使用新token进行前向传递，更新KV缓存，生成新的键和值。
   - 生成新的概率分布，用于采样下一个token。
3. **循环迭代：**
   - 重复步骤1和步骤2，直到达到生成的最大长度或满足停止条件。

下面详细解释这两个阶段KVCache的大小和更新方法，以及它们的不同之处。

预填充阶段的KVCache

大小和布局

在预填充阶段，KVCache的大小根据输入的上下文长度和模型层数确定。

1. **键和值的形状：**

$$\text{keys: } (\text{num_layers}, \text{batch_size}, \text{num_heads}, \text{seq_len}, \text{head_dim})$$

$$\text{values: } (\text{num_layers}, \text{batch_size}, \text{num_heads}, \text{seq_len}, \text{head_dim})$$

   其中：
   - `num_layers`：模型的总层数
   - `batch_size`：输入的批次大小
   - `num_heads`：多头注意力机制的头数
   - `seq_len`：输入序列的长度
   - `head_dim`：每个头的嵌入维度
2. **KVCache大小示例：**

   假设一个模型有 `12` 层，每层有 `16` 个头，每个头的维度为 `64`，批次大小为 `1`，上下文长度为 `512`，则KVCache的大小如下：
   - 键缓存大小：

$$(12, 1, 16, 512, 64)$$

   - 值缓存大小：

$$(12, 1, 16, 512, 64)$$

更新方法

预填充阶段，KVCache会随着输入的每个token进行填充和更新。

初始填充： 对于给定的输入上下文，模型计算每一层的键和值向量，并将它们存储到缓存中

解码阶段的KVCache

51CTO学堂

大小和布局

在解码阶段，KVCache的大小主要取决于上下文的长度和逐步生成的长度。

与预填充阶段不同的是，解码阶段的KVCache随着生成的序列不断增长。

1. **键和值的形状：**

$$\text{keys: } (\text{num_layers}, \text{batch_size}, \text{num_heads}, \text{max_seq_len}, \text{head_dim})$$

$$\text{values: } (\text{num_layers}, \text{batch_size}, \text{num_heads}, \text{max_seq_len}, \text{head_dim})$$

   其中：

   - `max_seq_len`：预填充序列长度 + 生成序列长度
2. **KVCache大小示例：**

   假设初始输入长度为 `512`，逐步生成长度为 `100`，则KVCache的最大长度为 `612`。

   - 键缓存大小：

$$(12, 1, 16, 612, 64)$$

   - 值缓存大小：

$$(12, 1, 16, 612, 64)$$

总结

- **KVCache的大小：**
  - 预填充阶段：缓存的大小取决于初始输入的长度。
  - 解码阶段：缓存的大小随着生成序列长度的增长而不断增加。
- **KVCache的更新方式：**
  - 预填充阶段：一次性将整个初始上下文填充到缓存中。
  - 解码阶段：逐步生成并更新缓存，每次生成新token时追加新的键和值。

逐步生成长度(Incremental Generation Length)指的是语言模型在解码阶段, 根据输入的初始文本(如提示词prompt), 一个token一个token地逐步生成输出序列的过程。

在解码开始时, 语言模型只有输入的初始文本,如"用英文写一首关于春天的诗"。然后模型根据这个输入, 预测并生成下一个最可能的token,如"Spring"。之后"Spring"会被添加到之前的输入中, 形成新的输入序列,模型再根据这个更新的输入去预测下一个token,如"is"。如此重复,直到满足某个停止条件(如遇到截止符<\s>, 或达到最大生成长度), 整个解码过程就结束了。

这个过程中, 生成的token序列(如"Spring is coming")的长度, 就被称为逐步生成长度。它一般远小于模型的最大序列长度限制。比如最大序列长度为2048的模型, 在实际解码时, 往往只需要生成几十到几百个token就可以得到满意的结果。

在解码阶段引入键值缓存(KV Cache), 主要就是为了提高处理这些逐步生成token的效率。随着解码的进行, 每生成一个新的token, 就动态地将其添加到KV Cache中, 更新键值对。这样模型就可以得到到不断增长的已生成序列, 从而实现循环机制。

所以总结来说, 逐步生成长度就是语言模型解码时动态生成token序列的长度, 反映了解码的进度。

合理设置逐步生成长度, 优化解码算法和KV Cache机制, 对于提升语言模型推理速度和生成质量至关重要。

总结

51CTO学堂

使用KV Cache，在token-by-token推理中：

- **初始token** 的计算复杂度为 $O(d^2)$。
- **后续每个token** 的计算复杂度为 $O(Nd)$，而不是重新计算整个序列，从而避免了二次计算复杂度。

其中 N 是序列长度，d 是每个序列元素的维度。

因此，通过使用KV Cache，可以将token-by-token推理的计算复杂度从 $O(N^2d)$ 降低到接近线性关系，

这种优化使得大模型能够处理更长的序列, 并在推理时显著提高了效率。

需要注意的是, 这里讨论的是推理阶段的计算复杂度。

在训练阶段, 由于需要对整个序列进行并行计算, 自注意力的计算复杂度仍然是 O(N^2)。

但是, 在实际应用中, 推理阶段的效率往往更为重要, 因为它直接影响了模型的响应速度和吞吐量。